

UNIVERSITY OF THE ARMED FORCES - ESPE

Department of Exact Sciences

Object - Oriented Programming

Homework05 - Slides Summary UML Reading

Verónica Daniela Ruiz Gámez

Jorge Edison Lascano

October 14, 2026

Titulo: _____ Fecha: ____/____/____

Summaries

OO Modeling Practices

Conceptual Modeling

Conceptual Modeling captures ideas and concepts to facilitate reasoning, analysis, design, and efficient communication regarding real-world systems

It can be applied at any point during the software development life cycle, design, analysis, specification, implementation, deployment.

UML (Unified Modeling Language): Collection of conceptual modeling languages

Case diagrams → Describing those who need a software system
Class diagrams → Describing the classes of objects that make up a software system
Interaction diagrams → Describing how objects interact
State charts → Describing the internal behavior of objects in a class

Loosely Coupled layers: Software should be organized into layers (such as User Interface, Application Logic, Persistent, Communications, and Operation System) where each layer provides clear abstractions to higher-level layers

Aggregation Over Inheritance: Aggregation should be preferred over inheritance when considerations are equivalent, as aggregation relationships are more flexible and can be modified at runtime. Composition serves as a stronger form of aggregation.

Program to an Interface or Abstraction: A class interface consists of its public methods and properties.

Class variables should generally be kept private because publicly accessible variables usually violate the object-oriented encapsulation principle.

Use identifiers that improve readability and Maintainability

- Choosing meaningful names improve readability and maintainability. Classes use nouns / nouns phrases in Pascal Casing (RightHandedWidget) variables use lowerCamelCasing based on their intended purpose, and methods use verb a verb phrases.

Testing with Executable Unit Test cases: Testing individual units with executable test, cases helps uncover design errors, verify correctness, document usage, and enable regression testing through testin frameworks.

Titulo: _____ Fecha: ___/___/___

Introduction to UML

Unified Modeling Language is a modeling language used in software engineering to visualize, specify, construct and document software systems

UML helps developers understand how a system works before implementation

UML is an expressive, semi-formal, process-independent and extensible object-oriented modeling language
It supports incremental development allowing elements to be hidden or left incomplete, and accommodating minor inconsistencies

Timeline for OO and UML

1960-1980: The birth, nurturing, and maturation of foundational object-oriented ideas, culminating in the widespread adoption of OOP Languages.

1990-Current: Marked by the "Method Wars", unification efforts, the standardization of UML, the rise of design patterns, and the modern deployment of full-life-cycle development tools.

UML Building Blocks and Diagrams

UML consists of modeling elements that build diagrams (sentences/paragraphs), which together form complete system models (papers/books)

Modeling Elements:
- Scoping
- Structural
- Behavioral
- Organizational
- Annotational
- Extension

Divided into definition/scope (use cases, automation boundaries), structural things (classes, objects, relations, nodes, interfaces, components), behavioral things (messages, states), organizational things (packages, views), annotations (comments), and extensions (stereotypes)

Types of diagrams

Structural Diagrams: Include class diagrams, object diagrams, component diagrams and deployment diagrams to display static architecture and design views

Behavioral Diagrams: Include use case diagrams, sequence diagrams, collaboration diagrams, state chart diagrams, and activity diagrams to capture dynamic behavior and system control flows.

- Class Diagrams: Commonly found in object-oriented modelling, they show classes, interfaces, collaborations, and their relationships to illustrate static design views.
- Object Diagrams: Show objects and the relationships to illustrate static snapshots of class instances. Show individual real or prototypical cases.

Titulo: _____ Fecha: ___/___/___

- Component Diagrams: Show components and their relationships to illustrate the static implementation views of a system.

- Deployment Diagrams: Show nodes and their relationships to illustrate the static deployment views of an architecture. Encloses one or more components.

Behavioral Diagrams

- Use Case Diagrams: Show use cases and actors to illustrate the use case view of a system.

- Sequence Diagrams: Interaction diagrams that emphasize the time ordering of messages sent and received by named or anonymous objects.

- Collaboration Diagrams: Interaction diagrams that emphasize the structural organization of communicating objects, showing objects, their links, and messages to illustrate dynamic views.

- Statechart Diagrams: Show state machines consisting of states, transitions, events and activities. They emphasize event-ordered behavior.

- Activity Diagrams: Show the flow of activities within a system, their ordering, and the objects that act or are acted upon.

Three Software-Engineering Modeling Perspectives

- Analysis Perspective: Focuses on problem-domain concepts and real-world objects to understand the system as it currently exists.

- Specification / High-Level Design Perspective: Focuses on solving problems, establishing automation boundaries, and defining what the system is to become using both real-world and software objects.

- Implementation Perspective: Focuses on construction using mostly software objects and solution-oriented concepts.

Software Development Activities and Best Practices

- Core Activities: Projects encompass primary phases and cross-cutting activities like conceptual modeling, coding, testing, and documentation.

Best Practices

- The primary purpose of diagram is to communicate, not just to look pretty.

- Avoid extraneous or redudant diagrams, Keep a balance between structural and behavioral elements. Give each diagram a meaningful name.

Titulo: _____ Fecha: ____/____/____

- Organize layout elements spatially to minimize crossing lines and place semantically close concepts physically close together

Basic Uses Cases

A use case describes a goal that a user or another external entity wants to accomplish with a system. It captures the functionality visible to the user without specifying the exact software implementation.

Actors: An actor is anything outside the system that interacts with it to achieve a goal. Actors may be People, Other software systems, Hardware

All user are actors, but no all actor are users

An actor represents a role, not necessarily a specific person. One person can play multiple roles, and multiple actors can participate in the same use case.

Use Cases vs System Interactions:

Use cases answer "What" and "Why" questions

System interactions describe the step-by-step sequence of how a user interacts with the system to fulfill a goal

Usec Case Diagrams and Relationships

Use case diagrams provide a visual way to documet user goals and functionality using four primary components.

- Actor Relationships:

Generalization ISpecialization: An actor set can be a subset of another actor set

- Use-Case Relationships & Dependencies:

- Extends: (<<extend>>) : Models a variation or argmentation of another use case. The extended use case can stand on its own, depends on it via specific extension points.

- Includes: (<<include>>) : Define how one use case requires the goal defined by another use case.

- Generalization: Define one use case as a subset of refinement of a general use case

Modeling Hints and Best Practices

- Finding Actor & Use Cases
- Keep diagrams focused on a single aspect of functionality with consistent detail levels.
- Minimize crossing lines

Titulo:_____ Fecha:__/__/__

Benefits

- Capture user-visible functions and identify who requires system functionality
- Document reuse opportunities through relationships and provide a basis for planning and system testing.

Objects, classes, and Basic Class Diagrams

Objects: They represent real-world "things" with a boundary in space an/or time, an abstraction, an identity, and encapsulated state and behavior. Concepts like events, relationships, behaviour, and constraints are typically not modeled as objects

Classes: These are abstractions that group collections of objects sharing common traits. Examples in a student records management system include Student, Course, Department, and Instructor

The Three Perspectives of Modeling

- Analysis Perspective: Describe the problem domain, real-word concepts, and sets of interacting objects

Specification Perspective: Provides a logical software description, defining component interfaces and service obligations

Implementation (Design) Perspective: Describe software components, data members, methods, and their technical deployment

Syntax for Classes, Object, Attributes, and Operation

- Class Syntax & Naming: Represented by a partitioned box containing the class name, attribute, operations, and responsibilities. Class names shoul be singular nouns written in Upper CamelCase.

Object Syntax: Represented as instanceName : ClassName with underlined name, displaying actual values for its attributes as instance of class.

Attributes: Define object properties or value placeholders. Syntax follows
[visibility] name [multiplicity] [: type] [= initial-value] [{property-string}]

Operations: Represent object interaction or methods. Syntax follows
[visibility] name [(parameter-list)] [: return-type] [{property-string}] using lowerCamelCase for names.

Titulo: _____ Fecha: ____/____/____

Types of Relationships in Class Diagrams

Class diagrams utilize several key relationship types:

- Generalization: Represents inheritance where a specialized class is a subset of generalized class
- Associations: Structural connections indicating that objects of one class are linked to object of another. They can be uni-directional, bi-directional or undefined, and can connect more than two classes via N-ary associations.
- Multiplicity Constraints: Placed near association end points to define how many objects can be linked
- Aggregation and Composition:
  - Aggregation represents an "is-part-of" relationship that is irreflexive, asymmetric, and transitive, denoted by an open diamond.
  - Composition is a stricter form of aggregation where the lifetime of the subpart is tightly bound to the aggregate object, denoted by a solid/black diamond

Dependencies: A "uses" relationship where a change to one component may cause changes in another, shown via a dashed line with an open arrowhead (common types include <<becomes>> and <<instance of>>).

Class Diagrams: Purpose and Pitfalls

Purpose: Class diagrams act like paragraphs in technical writing, showing the types of objects, properties, and relationships in a system similarly to Entity-Relationships models.

Common Pitfalls

- Focusing overly on static structure while ignoring dynamic behavior.
- Mixing different perspectives or levels abstraction within the same diagram.